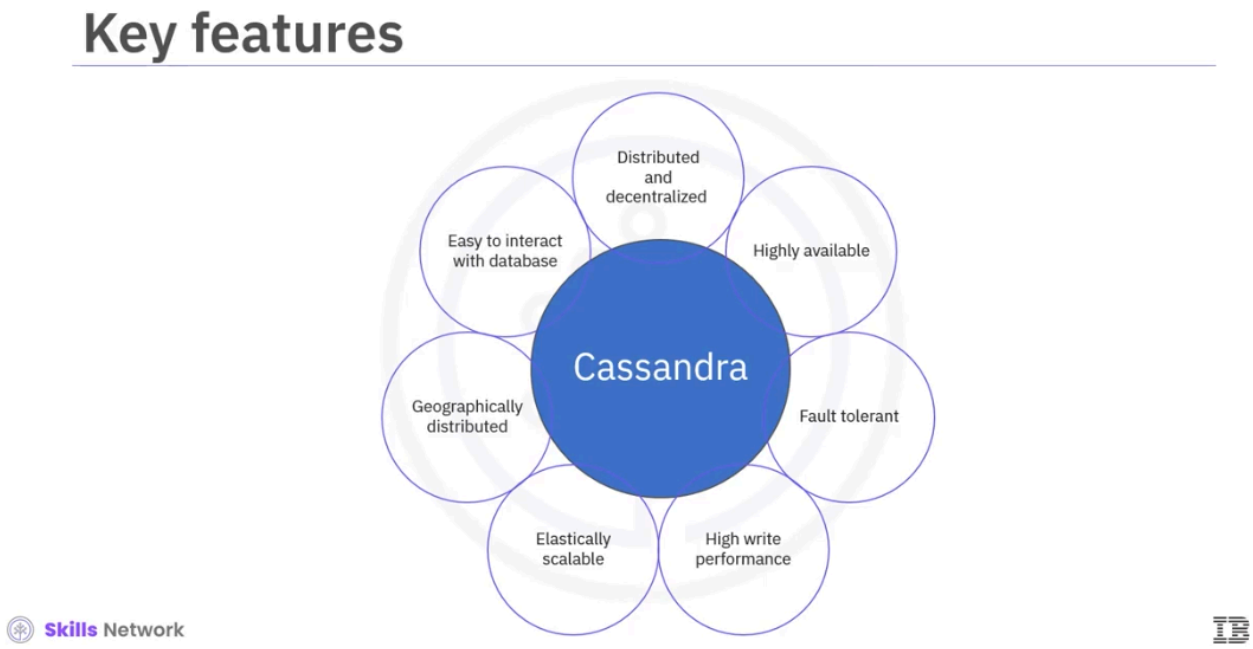
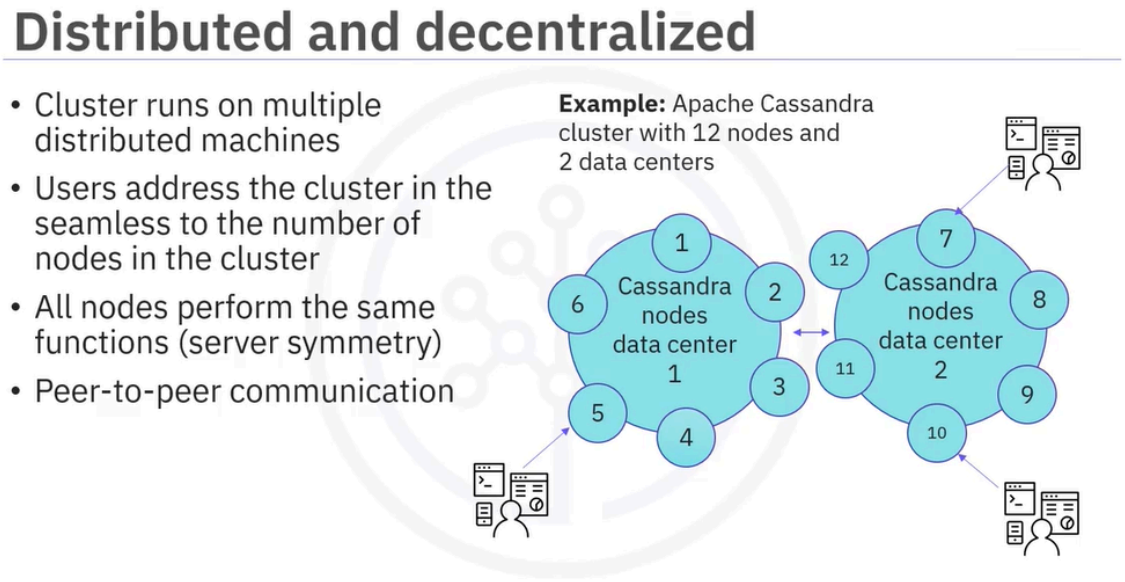
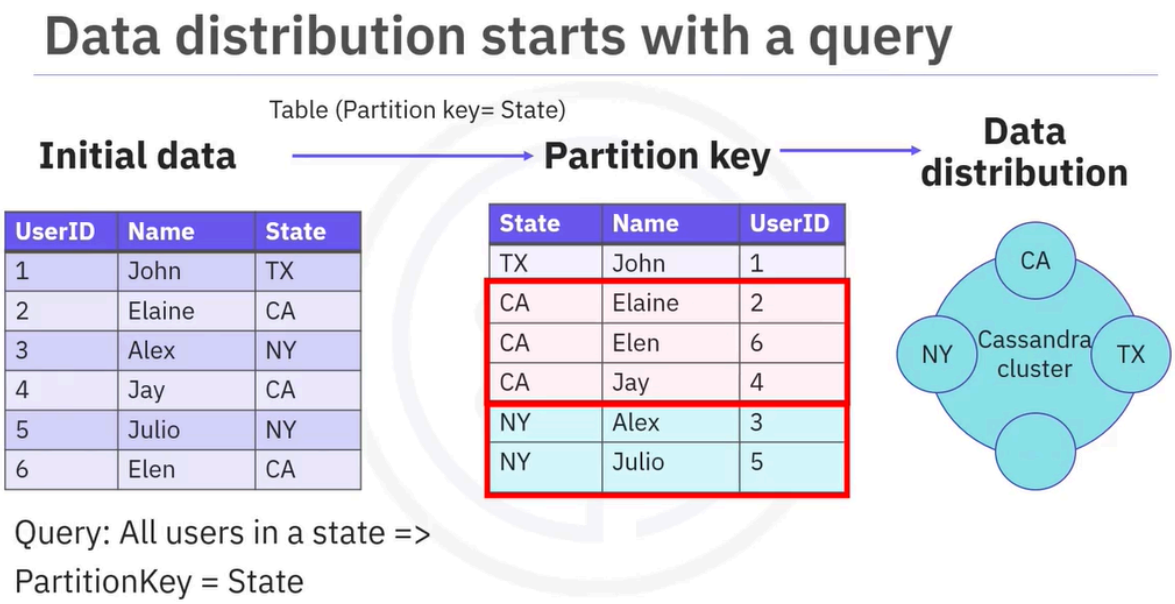
Key Features of Apache Cassandra

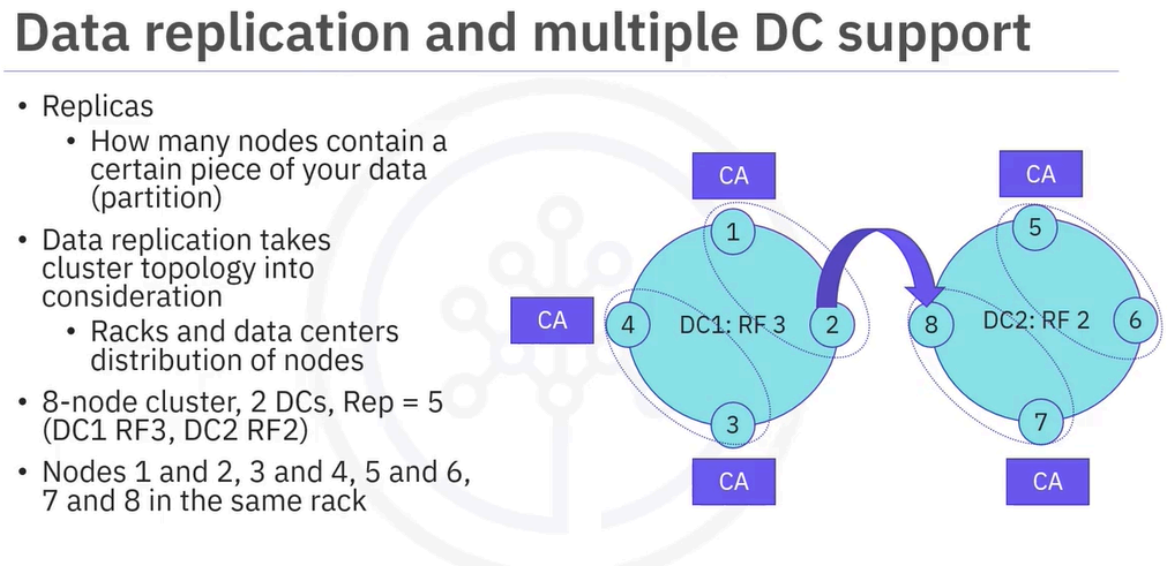




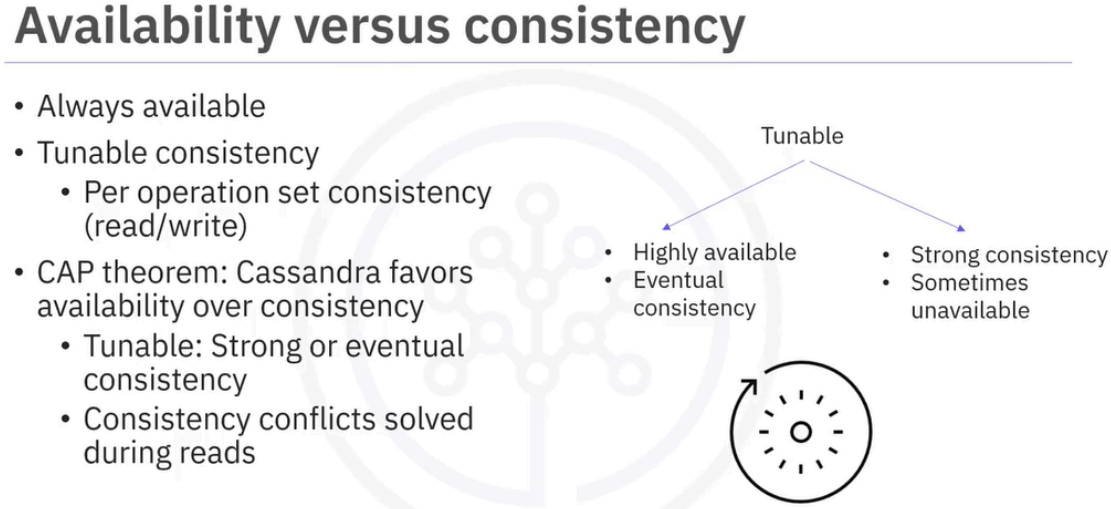
- While all NoSQL databases are distributed, you will not find many NoSQL databases that are both distributed and decentralized. Distributed means that Cassandra clusters can be run on multiple machines, while to the users and applications, everything appears as a unified whole. The architecture is built in such a way that the combination of Cassandra application client and server will provide sufficient information to route the user request optimally in the cluster.
- As an end user, you can write data to any of the Cassandra nodes in the cluster, and Cassandra will understand and serve your request. Decentralized means that every node in the Cassandra cluster is identical. That is, there are no primary or secondary nodes. Cassandra uses peer-to-peer communication protocol and keeps all the nodes in sync through a protocol called gossip.



- How does data end up in this distributed architecture? It all starts with the queries that are planned to be performed. Just to take a very simple example, you have some initial data containing user info and the state. If your queries are similar to, I would like to know about all users in a state, in this case, you need to group your data on the state column, and this is done by declaring a table that has the state column as the partition key. Just remember that Cassandra groups data based on your declared partition key and then distributes the data in the cluster by hashing each partition key called tokens. Each Cassandra node has a predefined list of supported token intervals, and data is routed to the appropriate node based on the key value hash and this predefined token allocation in the cluster



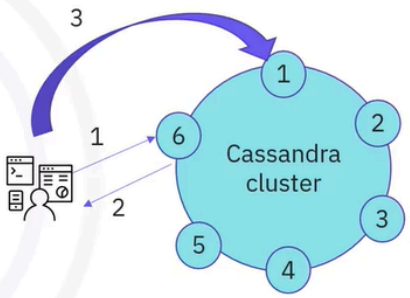
- After data is initially distributed in the cluster, Cassandra proceeds with replicating the data. The number of replicas refer to how many nodes contain a certain piece of data at a particular time. Data replication is done clockwise in the cluster, taking into consideration the rack and the data center's placement of the nodes.
- Data replication is done according to the set replication factor, which specifies the number of nodes that will hold the replicas for each partition. Let's take the data from the previous screen and try to distribute the California state data, partition CA, in an eight-node cluster, distributed in two data centers, DC1 with replication factor 3 and DC2 with replication factor 2. You can see in the diagram that some nodes are placed in the same rack. Cassandra will try to distribute data as much as possible between racks.



- One of the most important Cassandra features is its availability. Also, Cassandra is frequently referred to as eventual or tunable consistency in the sense that, by default, Cassandra trades consistency in order to achieve availability.
- But the good news is that developers can control exactly how much consistency they would like to have strong or eventual. As you may recall from the NoSQL introduction video earlier in the course, distributed systems cannot, according to CAP theorem, be consistent and available at the same time. Cassandra has been designed to always be available, meaning that if you lose a part of your cluster, there will still be nodes available to answer the service request, though the return data might be inconsistent. Consistency of the data can be controlled at the operation level, and it is tuned between strong consistency and eventual consistency. If data inconsistencies exist, these conflicts will be resolved during read operations. This is another unique Cassandra feature.

High availability and fault tolerance

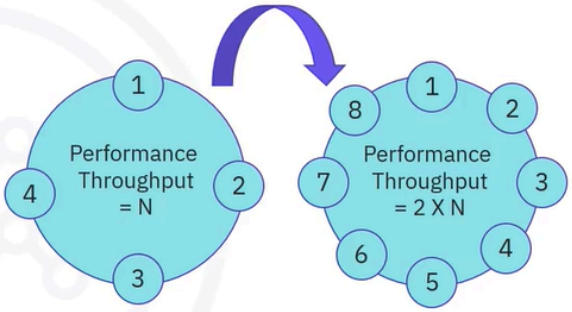
- Peer-to-Peer architecture
- The other nodes in the cluster immediately recognize nodes' temporary/permanent failures
- Nodes reconfigure the data distribution once nodes are taken out of the cluster
- Failed requests can be retransmitted to other nodes



- Fault tolerance is an inherent feature of the distributed and decentralized characteristic of Cassandra. The fact that all nodes have the same functions, communicate in a peer-to-peer manner, are distributed, and the data is replicated makes Cassandra a very tolerant and adaptable solution when nodes fail. The user contacts one node of the cluster. If the node is not responding, then the user will receive an error and contact another node.

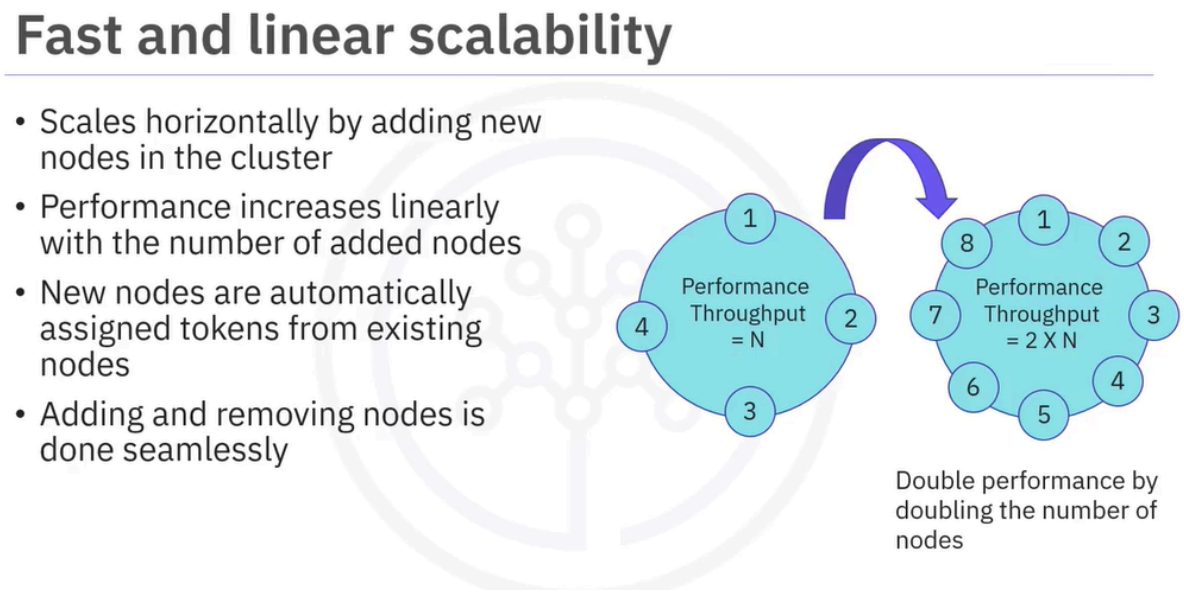
Fast and linear scalability

- Scales horizontally by adding new nodes in the cluster
- Performance increases linearly with the number of added nodes
- New nodes are automatically assigned tokens from existing nodes
- Adding and removing nodes is done seamlessly



Double performance by doubling the number of nodes

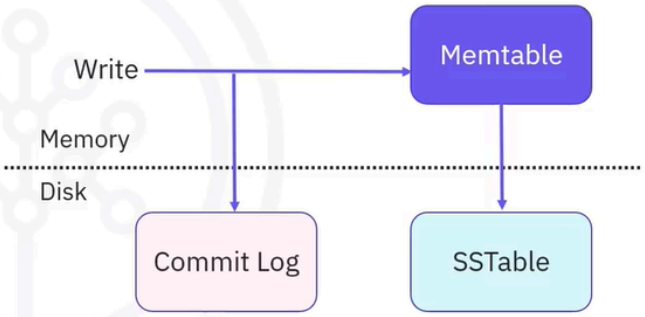
- The same architectural flexibility is visible in the way Cassandra scales the capability of the clusters. A cluster is scaled by simply adding nodes, and performance increases linearly with the number of added nodes. New nodes that are added immediately start serving traffic, while existing nodes move some of their responsibilities towards the new added nodes. Both adding and removing nodes is done seamlessly without interrupting cluster operations.



- Cassandra gracefully handles large numbers of writes first by parallelizing writes to all nodes holding a replica of your data.

High write throughput

- No reading before writing (by default)
- At node level
  - Writes are done in node memory and later flushed on disk
  - All disk writes are sequential, append-like operations



Write operations: node level

- One important Cassandra fact; by default, there's no read before write. At the node level, writes are performed in memory, meaning no read before and then flushed on disk. On disk, data is appended in a sequential manner with the data being reconciled later through compaction.

Cassandra query language (CQL)

Data Definition and Manipulation: CQL, an SQL-like syntax

```
CREATE TABLE test (
      groupid uuid,
      name text,
      occupation text,
      age int,
      PRIMARY KEY ((groupid), name)
);
INSERT INTO test (groupid, name, occupation, age)
            VALUES  (1001, 'Thomas', 'engineer', 24),
                                (1001, 'James', 'designer', 30),
                                (1002, 'Lily', 'writer', 35);
SELECT * FROM test WHERE groupid = 1001;
```

- Cassandra Query Language, or CQL for short, is the language used for data definition and manipulation. CQL syntax is similar to SQL syntax, thereby

reducing the time a developer needs to get started with Cassandra. Operations like create table, insert, update, delete, alter, and more can be used in CQL. Be aware that while syntax-wise, there are similarities between CQL and SQL, the resemblance stops here. The ways write and read operations are executed in Cassandra are different than how these are executed in relational databases.

Recap

In this video, you have learned that:

- Cassandra's distributed and decentralized architecture enables availability, scalability, and fault-tolerance
- Data distribution and replication happen in one or more data center clusters
- Cassandra provides high write throughput
- CQL is the language used to communicate with Cassandra